
Table of Contents

PREAMBLE	1
=====	=====
1. COMMENTING	1
=====	=====
=====	=====
2. ECHO AND FOLDOVER	4
=====	=====
=====	=====
3. IMAGE FILTERING	6
=====	=====
=====	=====
4. AI FOR SAMPLING	11
=====	=====
=====	=====
ALL FUNCTIONS SUPPORTING THIS CODE %%	16

PREAMBLE

DO NOT REMOVE THE LINE BELOW

```
clear; close all;
```

```
=====
=====
```

1. COMMENTING

```
=====
=====
```

MAKE SURE FILE BELOW IS IN SAME DIRECTORY AS THIS FILE

```
type('eel3135_lab04_comment.m')
```

```
% Convolution can be seen as the mathematical expression of how one signal
% can modify the other, you will need to comment the answers to the
% questions on the following lines of code to demonstrate the basics of
% convolution. Also answer the question at the end in order to understand
% the effects that convolution may have on the original signal.
```

```
%% ONE-DIMENSIONAL CONVOLUTION
```

```
% INPUT SIGNAL
```

```

xx = [1 1 -1 -1 1 1 -1 -1 1 1 1 1 -1 -1 -1 -1];
% <-- Answer: What is the length of xx?
% 16

% FILTER COEFFICIENTS / IMPULSE RESPONSE
bk = [1/4 1/4 1/4 1/4];
% <-- Answer: What is the length of bk?
% 4

% FILTER OUTPUT
yy = conv(bk, xx);
% <-- Answer: What is the length of yy?
% L + N - 1 = 16+4-1 = 19

% PERFORM FILTERING IN ALTERNATIVE WAY
zz = 1/4*shift(xx, 0) + 1/4*shift(xx, 1) + 1/4*shift(xx, 2) + 1/4*shift(xx,
3);
% <-- Answer: What is the length of zz?
% 16

% --> ANSWER BELOW: Explain why yy and zz are different lengths. <--
% yy is computed using full linear convolution, which produces an output
% of length N + M - 1 because it includes all boundary overlaps between
% the signal and the filter.
%
% In contrast, zz is formed using the custom shift() function, which
% forces the shifted signals to remain the same length as xx.
%
% As a result, zz truncates boundary values while yy includes them,
% causing yy to be longer than zz.

% PLOT
figure(1)
subplot(411)
stem(xx); axis([0 20 -1 1])
ylabel('Amplitude')
subplot(412)
stem(bk); axis([0 20 -1 1])
ylabel('Amplitude')
subplot(413)
stem(yy); axis([0 20 -1 1])
ylabel('Amplitude')
subplot(414)
stem(zz); axis([0 20 -1 1])
ylabel('Amplitude')
xlabel('Samples')

%% TWO-DIMENSIONAL CONVOLUTION

% INPUT SIGNAL
x2 = [ones(15) -1*ones(15)]*255;

```

```

% FILTER COEFFICIENTS
b2 = (1/8)*[0 1 1 1 1 0; 0 1 1 1 1 0];
b3 =      [1 1 1 -1 -1 -1; 1 1 1 -1 -1 -1];

% OUTPUT
y2 = conv2(x2,b2);
y3 = conv2(x2,b3);

% --> ANSWER BELOW: Why does filter b2 affect the image as it does?
%      What are possible applications of filter b2? <--

% Filter b2 acts as a smoothing filter because its positive
% coefficients average neighboring pixel values, which blurs the sharp
% vertical boundary in the image.
% It is commonly used for noise reduction, image smoothing, or as a
% preprocessing step before edge detection.
% Additionally, conv2 performs full convolution with zero-padding outside
% the image, which causes the left boundary to darken slightly when the
% filter averages white pixels with zeros.

% --> ANSWER BELOW: Why does filter b3 affect the image as it does?
%      What are possible applications of filter b3? <--
% Filter b3 compares neighboring regions using positive weights on one side
% and negative weights on the other, producing a directional intensity
% difference.
% This emphasizes transitions between bright and dark areas while uniform
% regions
% cancel toward zero due to the filter's zero-sum structure.
% Such filters are commonly used for edge detection, boundary enhancement,
% and gradient-based feature extraction.
% (I had to do further research about this one to help build my knowledge
% because I was really confused by the results, and I still am.
% If it is supposedly an edge detection type filter,
% wouldn't the middle have a very clear edge that it should have detected
% or at least done something with?)

% PLOT THE FIRST FILTER RESULTS
figure(2)
subplot(231)
image(x2)
xlabel('x'); ylabel('y'); zlabel('z');
title('x2')
axis equal; axis tight; colormap('gray');
subplot(232)
image(b2)
xlabel('x'); ylabel('y'); zlabel('z');
title('b2')
axis equal; axis tight; colormap('gray');
subplot(233)
image(y2)
xlabel('x'); ylabel('y'); zlabel('z');
title('y2')

```

```

axis equal; axis tight; colormap('gray');

% PLOT THE SECOND FILTER RESULTS
subplot(234)
image(x2)
xlabel('x'); ylabel('y'); zlabel('z');
title('x2')
axis equal; axis tight; colormap('gray');
subplot(235)
image(b3)
xlabel('x'); ylabel('y'); zlabel('z');
title('b3')
axis equal; axis tight; colormap('gray');
subplot(236)
image(y3)
xlabel('x'); ylabel('y'); zlabel('z');
title('y3')
axis equal; axis tight; colormap('gray');

function xs = shift(x, s)
%SHIFT Takes existing signal x, and shifts it by s time units

% Declares new vector that will be shifted over
xs = zeros(length(x), 1);

for n = 1:length(x)
    % Determines whether index we are accessing is within bounds
    if n-s > 0 && n-s < length(x)
        % Shifts over x by s time units
        xs(n) = x(n-s);
    end
end
end
end

```

```

=====
=====

```

2. ECHO AND FOLDOVER

```

=====
=====

```

MAKE SURE FILE BELOW IS IN SAME DIRECTORY AS THIS FILE Audio From: <https://freemusicarchive.org/music/gregor-quendel/> Title: Bortolazzi, Bartolomeo - Notturmo - Periodical Amusements fo Author: Gregor Quendel

```
[x, fs] = audioread('Gregor Quendel - Bortolazzi - short.wav');
```

```

% -----
% 2(a)
% -----
% Only need to modify function -- this area can be empty
% Create the function y = echo(x, s, A) that outputs the following signal:
%  $y[n] = x[n] + Ax[n-s]$  ,
% where A is the echo amplitude and s is the echo delay. You may want to use
the shift
% function in Question #1. Include the echo function at the end of the
skeleton file.

% -----
% 2(b)
% -----
% Save (as echo.wav) the output from applying echo to the provided x (in the
skeleton code)
% with s = 20000 samples and amplitude to A = 0.5

s = 20000;      % delay in samples
A = 0.5;        % echo amplitude

y_echo = echo(x, s, A);

audiowrite('echo.wav', y_echo, fs);

% -----
% 2(c)
% -----
% ANSWER QUESTION BELOW
% Answer in your comments: If our sampling frequency is 44100 samples per
second, and
% we are delaying by 20000 samples, how long is the delay on the signal in
seconds?
%
% Delay time = samples / sampling frequency
% Delay = 20000 samples  $\times$  (1 sec / 44100 samples)  $\approx$  0.4535147392 seconds

% -----
% 2(d)
% -----
% Only need to modify function -- this area can be empty
% Create a function y = foldover(x, A) that outputs the following signal:
%
%  $y[n] = x[n]$  when  $-A \leq x[n] \leq A$ 
%  $= 2A - x[n]$  when  $x[n] > A$ 
%  $= -2A - x[n]$  when  $x[n] < -A$ 
% where A is an amplitude.

```

```

% -----
% 2(e)
% -----
A = 0.1;    % foldover threshold

y_fold = foldover(x, A);

audiowrite('foldover.wav', y_fold, fs);

% -----
% 2(f)
% -----
% ANSWER QUESTION BELOW
% Answer in your comments: How does the foldover distortion affect the audio
% and why?
% Show this mathematically.
%
% Foldover distortion is nonlinear and "reflects" samples that exceed ±A
% back into the range,
% so peaks are not clipped flat; instead their sign/slope can invert, which
% audibly adds harsh,
% metallic distortion and many new harmonics.
%
% Mathematically, for A = 0.1: if x[n] = 0.18 (>A), then y[n] = 2A - x[n] =
% 0.2 - 0.18 = 0.02
% (a large positive peak folds back down). If x[n] = -0.25 (<-A), then y[n]
% = -2A - x[n]
% = -0.2 - (-0.25) = 0.05 (a negative peak folds upward).
%
% Because y[n] depends on the amplitude of x[n] via this piecewise rule (not
% a linear scaling),
% the waveform shape changes and new frequency components are created,
% producing distortion.
%

```

```

=====
=====

```

3. IMAGE FILTERING

```

=====
=====

```

```

img = imread('ai_programming_dragon_small.png');

```

```

% -----

```

```

% 3(a)
% -----
% Plot side-by-side the input (found in the skeleton code) and output of the
% filter defined below.
% After applying the filter, normalize the image by subtracting its minimum
% value and then
% sequentially dividing the maximum value by 255. This should force its
% value to range 0 to
% 255. Refer to the output of these operations as f1[u,v]. Use imagesc and
% subplot to plot
% the image before and after applying the processing.

g = double(img);

% Filter kernel
w = [-2 -1 0;
      -1 1 1;
       0 1 2];

% 2D convolution
f = conv2(g, w, 'same');

% Normalize to range 0-255:

% Subtract minimum
f1 = f - min(f(:));

% Divide by (maximum / 255)
f1 = f1 / (max(f1(:)) / 255); %same as f1 = 255 * (f1 / max(f1(:)));

% Plot input and output side-by-side
figure;
subplot(1,2,1)
imagesc(img); axis image; colormap gray;
title('Input Image')

subplot(1,2,2)
imagesc(f1); axis image; colormap gray;
title('Filtered + Normalized (f1)')

% -----
% 3(b)
% -----
% ANSWER QUESTION BELOW
% Answer in your comments: Does this filter blur, sharpen, or extract edges
% in the image?
% How / why do you know that from the impulse response / kernel?
%
% This filter primarily extracts edges, not blurring.

```

```

% I know that from the impulse response/kernel because it has both negative
and positive
% coefficients arranged asymmetrically (negative weights on the top-left and
positive
% weights on the bottom-right), which computes a directional intensity
difference and
% responds strongly at transitions while flatter regions change much less.
% Also, the coefficients sum to +1, so it preserves some DC/overall
brightness
% while still boosting edges.

```

```

% -----
% 3(c)
% -----
% Plot side-by-side the input f1[x,y] and output f2[x,y] of the thresholding
system defined
% below. Use imagesc and subplot.

```

```

% f2[u,v] = 1 if f1[u,v] > 140
%           0 otherwise

```

```

% Apply threshold
f2 = zeros(size(f1));
f2(f1 > 140) = 1;

```

```

% Plot side-by-side
figure;
subplot(1,2,1)
imagesc(f1); axis image; colormap gray;
title('Input (f1)')

```

```

subplot(1,2,2)
imagesc(f2); axis image; colormap gray;
title('Thresholded Output (f2)')

```

```

% -----
% 3(d)
% -----
% Plot side-by-side the input f2[x,y] and output of the filter defined
below. Use imagesc and
% subplot.

```

```

% Averaging filter
w2 = (1/9) * ones(3,3);

```

```

%same as:
% w2 = [1/9 1/9 1/9;
%       1/9 1/9 1/9;
%       1/9 1/9 1/9];

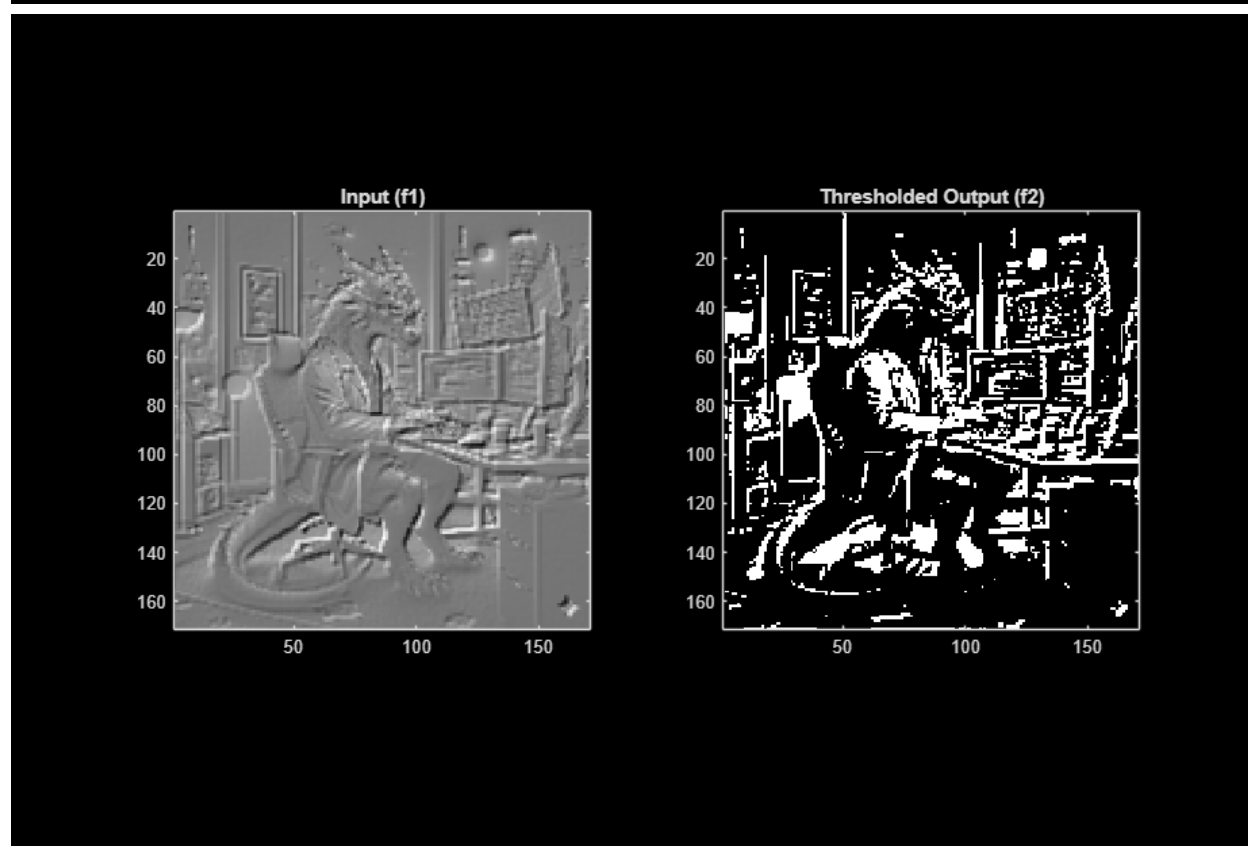
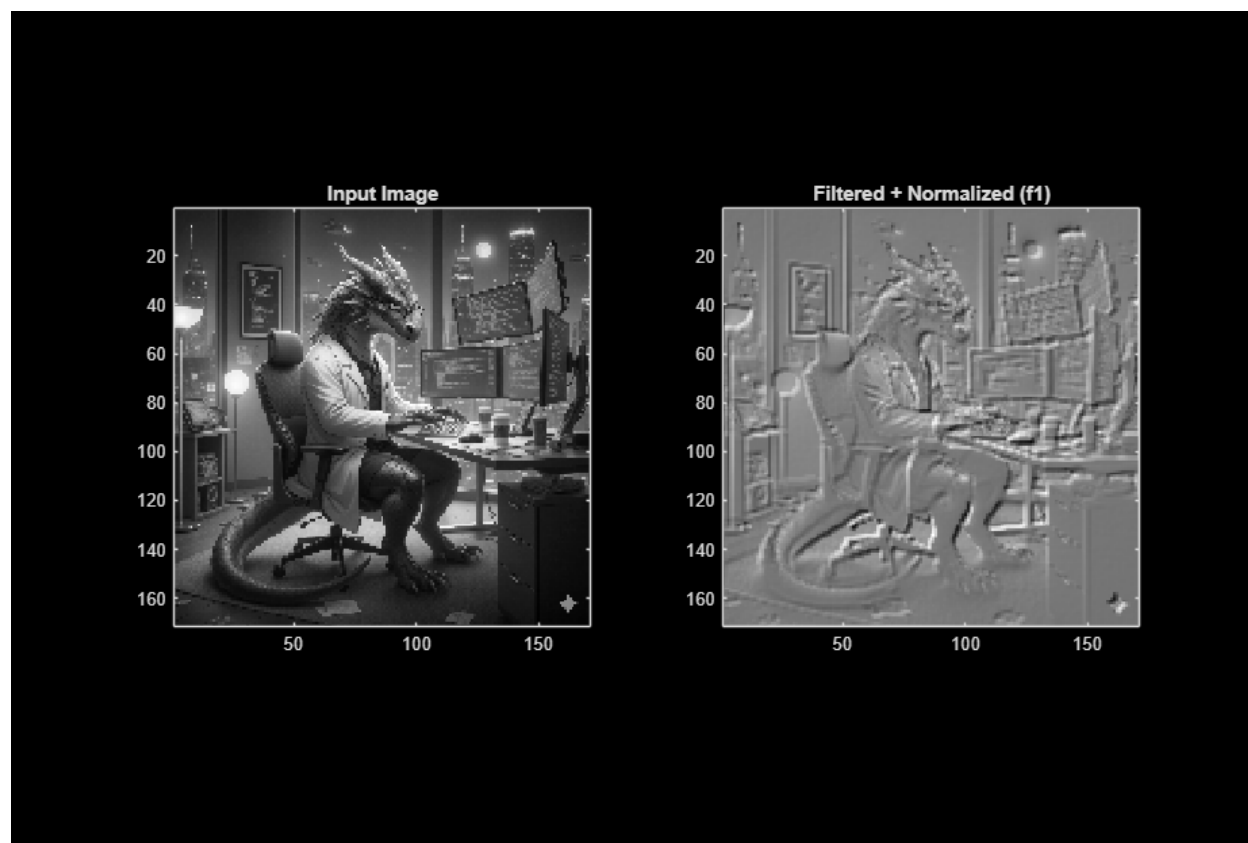
```

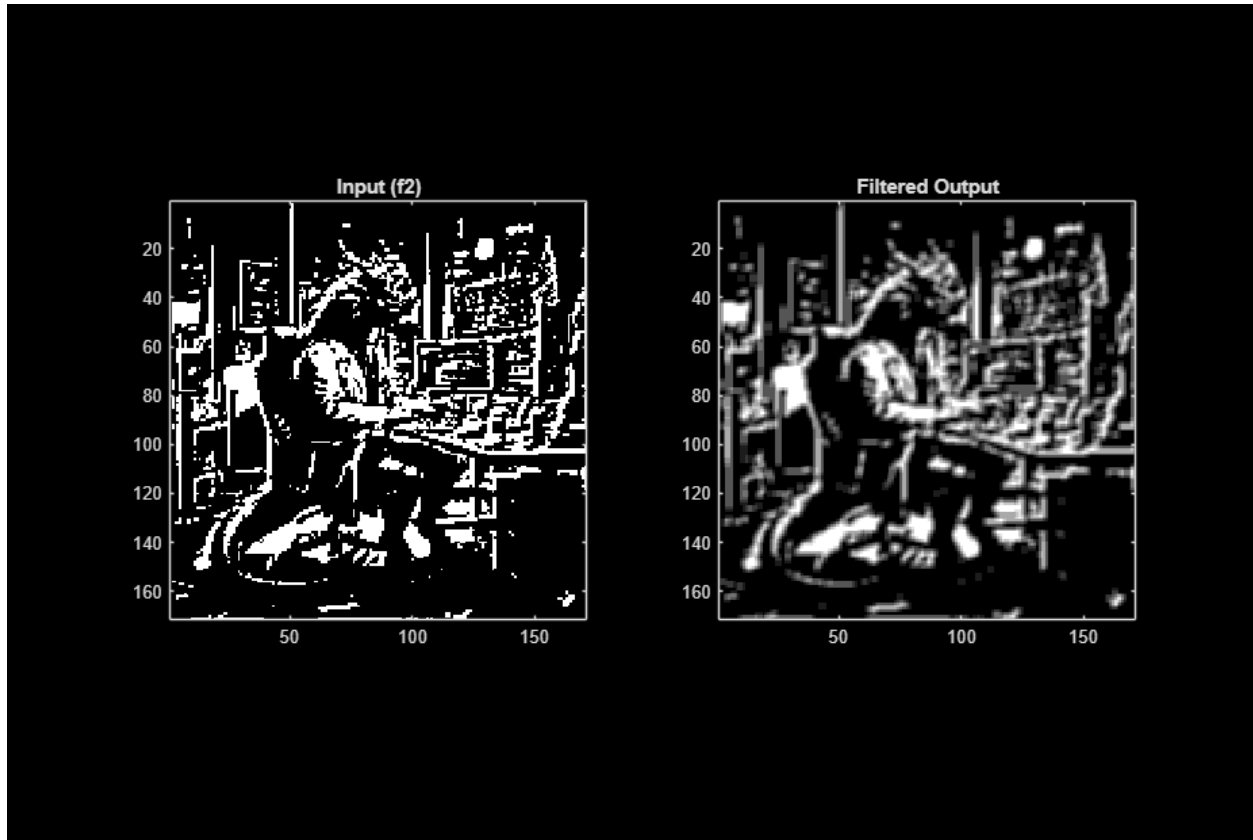
```
% convolution
f3 = conv2(f2, w2, 'same');

% Plot side-by-side
figure;
subplot(1,2,1)
imagesc(f2); axis image; colormap gray;
title('Input (f2)')

subplot(1,2,2)
imagesc(f3); axis image; colormap gray;
title('Filtered Output')

% -----
% 3(e)
% -----
% ANSWER QUESTION BELOW
% Answer in your comments: Does this filter blur, sharpen, or extract edges
% in the image?
% How / why do you know that from the impulse response / kernel?
%
% This filter blurs/smooths the image. I know that from the impulse
% response/kernel because all the coefficients are positive and equal, and
% sum to 1, which means each output pixel is the average of its
% 3x3 neighborhood. Averaging reduces sharp intensity transitions,
% producing a smoothing/blur effect.
```





```
=====
=====
```

4. AI FOR SAMPLING

```
=====
=====
```

----- 4(a) ----- I was unsure if I should put the function here or at the bottom, since there was not a comment telling me I can leave this section blank. I put it at the bottom because of this direction on canvas: When creating new functions: – Place them at the end of the skeleton M-file, unless otherwise specified

```
% -----
% 4 (b)
% -----
% Description/prompt:

% I need MATLAB code for Question 4 of my EEL 3135 lab.
% Please help me design a function:
%
% output_img = ai_filter(img)
```

```

%
% Requirements:
%
% The input img will be a grayscale image
% The filter must be implemented ONLY using 2D convolution
% You may choose any meaningful filter behavior (edge enhancement,
sharpening, smoothing, artistic effect, etc.)
% The function should return the filtered image as output_img
% Keep the implementation consistent with prior labs (clear structure,
comments, readable MATLAB style)


% First clarification question
%
% When we design a 2-D convolution filter, the kernel (mask) determines the
visual effect.
%
% Smoothing kernels (e.g., averaging) reduce noise but blur edges.
% Edge-enhancement / sharpening kernels (e.g., Laplacian) highlight
transitions.
% Artistic kernels (e.g., emboss) create stylized textures.
% Which overall effect would you like the ai_filter to produce? (Choose one
or describe a custom goal.)
%
%
% Sharpening / edge-enhancement


% Second clarification question
%
% The spatial extent of the convolution kernel controls how many neighboring
pixels influence each output sample.
%
% A 3×3 kernel uses the immediate 8 neighbors - fast, low memory, good for
basic sharpening.
% A 5×5 kernel reaches farther, allowing stronger edge emphasis or combined
smoothing-sharpening, but costs more computation.
% What kernel size would you prefer for the sharpening filter? (Enter 3 or 5.)
%
%
% 5


% Third clarification question
%
% Convolution must define how pixels outside the image border are treated.
%
% Zero-padding inserts zeros - can create dark halos.
% Replication repeats the edge pixel value - preserves overall brightness.
% Symmetric (mirror) reflects the image - reduces edge artifacts.
% Which border-handling method should the filter use? (Enter zero, replicate,
or symmetric.)
%
%
% symmetric

```

```

% Fourth clarification question
%
% The strength of the sharpening can be adjusted by scaling the
high-frequency component.
%
% Standard Laplacian (kernel sum=0) gives modest edge boost.
% High-boost adds a multiple  $\alpha > 1$  of the original image to the Laplacian
result, giving stronger emphasis.
% What sharpening strength would you like?
% Enter standard for a basic Laplacian, or provide a numeric
gain  $\alpha$  (e.g., 1.5, 2, 3).
%
%
% 1.5

% Fifth clarification question
%
% After convolution the pixel values may fall outside the usual 8-bit
range [0,255].
%
% Clip  $\rightarrow$  force any value  $< 0$  to 0 and any  $> 255$  to 255.
% Rescale  $\rightarrow$  linearly map the full output range back to [0,255].
% No-adjust  $\rightarrow$  return the raw floating-point result (useful for further
processing).
% Which post-processing should the function apply to the filtered image?
(Enter clip, rescale, or none.)
%
% rescale

% Design Summary - 5  $\times$  5 High-Boost Sharpening Filter
%
% Design element      Choice
% Effect              Edge-enhancement / sharpening
% Kernel size         5  $\times$  5
% Border handling      Symmetric (mirror) padding
% Sharpening strength   High-boost with gain  $\alpha = 1.5$ 
% Output post-processing Linear rescaling to [0, 255]

%after this it provided equations and the code, which I pasted in a
%function at the bottom.

% -----
% 4(c)
% -----

img = imread('ai_programming_dragon_small.png');

% Apply AI filter
output_img = ai_filter(img);

% Recreate kernel K for visualization

```

```
L = [ 0  0 -1  0  0;
      0 -1 -2 -1  0;
     -1 -2 16 -2 -1;
      0 -1 -2 -1  0;
      0  0 -1  0  0];

alpha = 1.5;
delta = zeros(5);
delta(3,3) = 1;

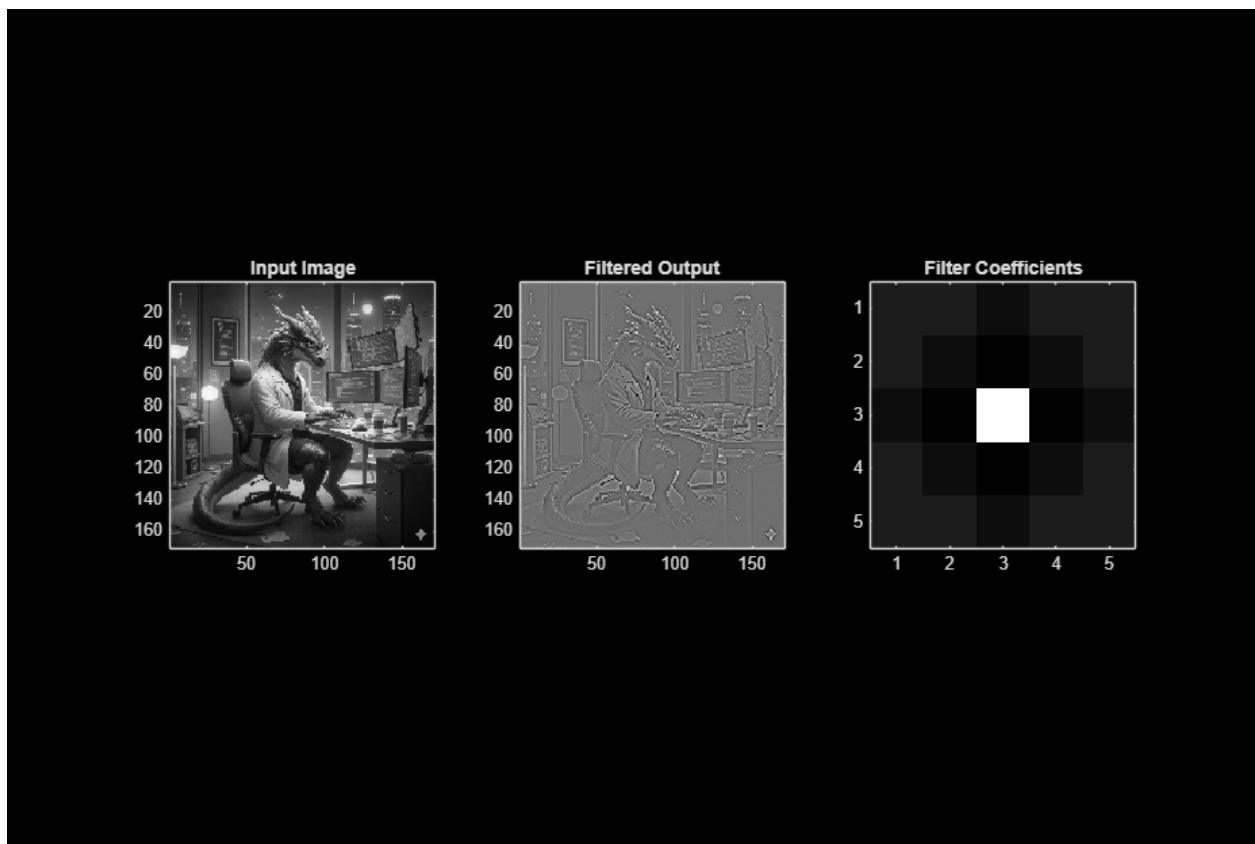
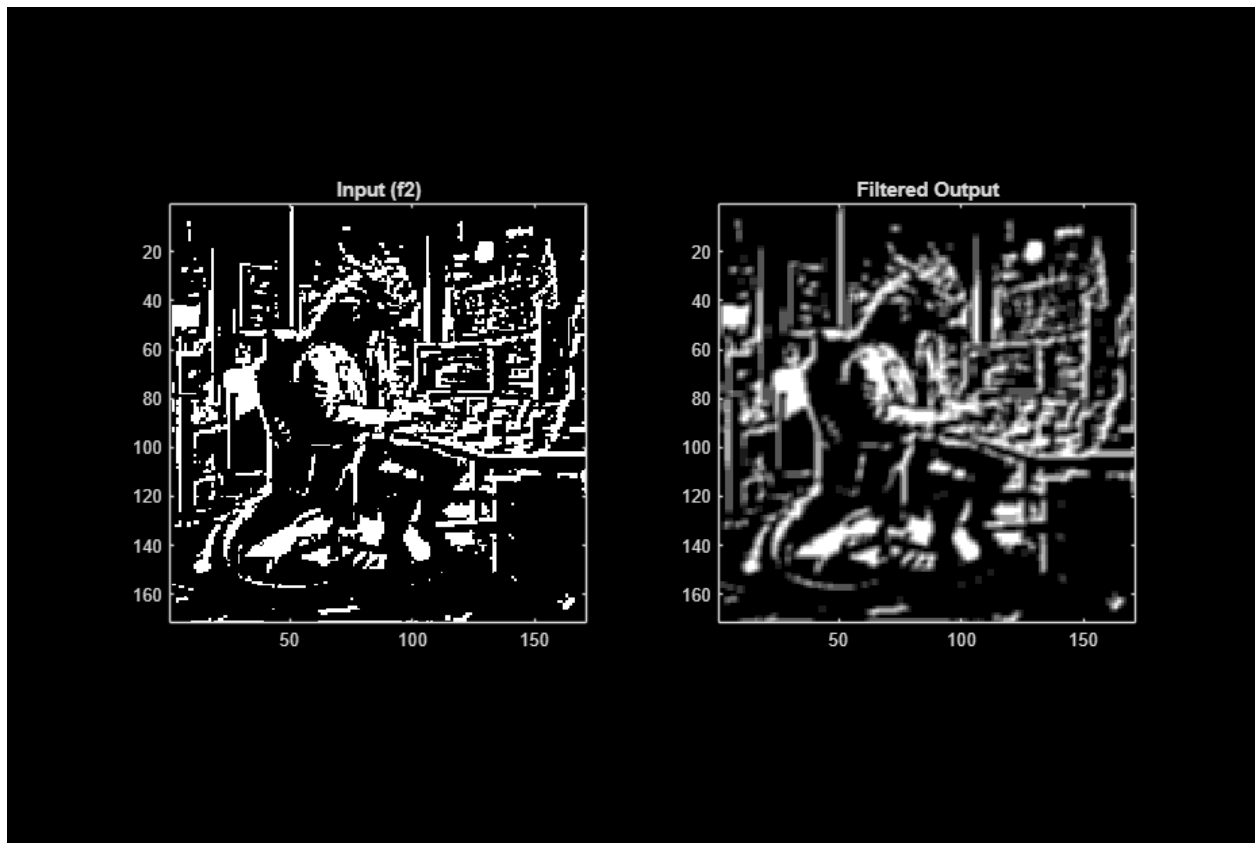
K = (alpha-1)*delta + alpha*L;

figure;

subplot(1,3,1)
imagesc(img);
axis image;
colormap gray;
title('Input Image')

subplot(1,3,2)
imagesc(output_img);
axis image;
colormap gray;
title('Filtered Output')

subplot(1,3,3)
imagesc(K);
axis image;
colormap gray;
title('Filter Coefficients')
```



ALL FUNCTIONS SUPPORTING THIS CODE % %

```
function y = echo(x, s, A)
%ECHO    Adds an echo to signal x with delay s and amplitude A
%        y[n] = x[n] + A*x[n-s]

    % make sure column vector to match shift output
    % x = x(:);

    % Delayed version of the signal
    x_delayed = shift(x, s);

    % Echo output
    y = x + A * x_delayed;

end

function y = foldover(x, A)
%FOLDOVER    Foldover distortion with A
%            y[n] = x[n]                when -A <= x[n] <= A
%            = 2A - x[n]                when x[n] > A
%            = -2A - x[n]               when x[n] < -A

    y = x; % start with the unchanged signal

    idx_hi = (x > A);
    idx_lo = (x < -A);

    y(idx_hi) = 2*A - x(idx_hi);
    y(idx_lo) = -2*A - x(idx_lo);

end

function xs = shift(x, s)
%SHIFT    Takes existing signal x, and shifts it by s time units

    % Declares new vector that will be shifted over
    xs = zeros(length(x), 1);

    for n = 1:length(x)
        % Determines whether index we are accessing is within bounds
        if n-s > 0 && n-s <= length(x)
            % Shifts over x by s time units
            xs(n) = x(n-s);
        end
    end

end

end
```

```
function output_img = ai_filter(img)
    % 1. Convert to double for arithmetic
    img_d = double(img);

    % 2. Define 5x5 Laplacian
    L = [ 0  0 -1  0  0;
          0 -1 -2 -1  0;
         -1 -2 16 -2 -1;
          0 -1 -2 -1  0;
          0  0 -1  0  0];

    % 3. High-boost kernel ( $\alpha = 1.5$ )
    alpha = 1.5;
    delta = zeros(5); delta(3,3) = 1;      % impulse
    K = (alpha-1)*delta + alpha*L;          % combined kernel

    % 4. 2-D convolution with symmetric padding
    raw_out = conv2(img_d, K, 'same');      % or
    imfilter(img_d,K,'symmetric','same')

    % 5. Linear rescaling to [0,255]
    minVal = min(raw_out(:));
    maxVal = max(raw_out(:));
    out_scaled = 255 * (raw_out - minVal) / (maxVal - minVal);

    % 6. Return uint8 image
    output_img = uint8(out_scaled);
end
```

Published with MATLAB® R2025b